

IDONTKNOWITSNAME

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

We present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis and compiler-level optimizations to expose binding lifetimes, expose application-level structure and proactively re-layout prompts to improve cache reuse. The serving runtime exploits the heterogeneity information to optimize KV-cache behavior across requests by prefetching, protection and eviction, according to the predicted future utility of cache entries. We implemented IDONTKNOWITSNAME on SGLang and evaluated it against multiple established serving systems. Our results shows that IDONTKNOWITSNAME.....

1 Introduction

Agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. Systems such as ByLLM [3], NVIDIA Object-Oriented Agents (NOOA) [5], and DSPy [9] allow developers to describe model behavior using functions, objects, signatures, and modules, which framework runtimes lower into concrete prompts. This approach also appears in systems such as SIGIL [4], which compiles natural-language skill documents into programs containing typed LLM calls. LLM requests from these frameworks are generated programmatically: they are the serializations of program state rather than free text.

We observed that these programmatically generated LLM requests are inherently heterogeneous. The heterogeneity arises from three dimensions: (1) Sharing scope: one value can be shared across multiple invocations, or only within a single invocation. (2) Lifetime: Different bindings have different lifetimes. Some bindings are single-shot volatile inputs, while others may be reuse across the whole session. (3) Evolution pattern: context for LLM calls can evolve differently overtime, such as remaining fixed, being replaced, or accumulating across invocations. Figure 1 illustrates this heterogeneity in a HoVer-style [8] fact check agent. The claim persists throughout a verification session, the evidence list accumulates over time, and each search result is consumed

once and replaced. Although these values appear in the same model invocation, they offer substantially different reuse opportunities.

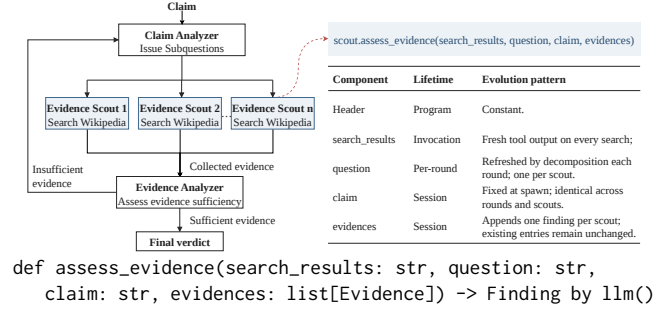


Figure 1. Heterogeneity within a HoVer-style [8] Fact Check Agent’s generated prompt for the summary-making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against the accumulated evidence and search results, then answers it.

This distinction matters because main-stream KV-cache serving backbones are prefix-based. In the example, placing the accumulating evidences binding after the short-lived `search_results` binding prevents the model from reusing the evidence prefix whenever the search result changes. The model must therefore recompute KV states for tokens whose contents remain unchanged. Moreover, without knowing which components persist or evolve, the serving system cannot accurately determine which cached tokens should be retained, evicted, or prefetched.

However, existing agent serving systems primarily optimize at the level of requests or agent invocation sequences. Pythia [16] profiles agent workflows to guide cache management, request scheduling, and autoscaling. CacheScout [17] learns execution transitions online to guide eviction and prefetching, while PBKV [18] predicts future invocations and estimates the reuse potential of cached requests. These systems exploit predictability between invocations, but treat the contents of each prompt as an opaque token sequence. Consequently, they cannot identify the heterogeneity within a single prompt, leading to a suboptimal serving performance for such program-integrated agents.

To close the gap, we present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME achieves this by exposing additional prefix reuse and keeping the resulting KV states resident when they are likely to be needed. The identification and exploitation of prompt heterogeneity introduces the following two challenges:

First, heterogeneity is not always explicit in prompt. The sharing scope and lifetime of bindings are closely bonded with the program-level structure and actual runtime assignment. Determining which layout best serves prefix caching is therefore not trivial. IDONTKNOWITSNAME combines static analysis with runtime information to determine prompt layouts. The compiler derives a partial order over prompt components based on their sharing scopes and lifetimes. Then, IDONTKNOWITSNAME uses the actual token lengths of bindings observed from log to order components left unconstrained by this partial order, producing a concrete layout that improves prefix cache efficiency.

Second, exploiting future reuse requires coordinating cache retention and prefetching. Prompt components differ in when they will be reused and how much recomputation their cached prefixes can avoid. Retaining these prefixes consumes capacity needed by other requests, while prefetching too early occupies device memory and prefetching too late fails to hide transfer latency. Inappropriate KV-cache eviction would lead to unnecessary recomputation. IDONTKNOWITSNAME addresses this challenge through two coordinated planners: a protection planner prioritizes cached prefixes according to their expected reuse and benefit, while a prefetch planner schedules transfers based on predicted invocation deadlines and available device memory.

We implemented IDONTKNOWITSNAME on SGLang [19].... We evaluated IDONTKNOWITSNAME with three representative agent workloads: HoVer-style [8] fact check agent, HumanEval [2] code agent, and a BFCL [13] tool-use agent.... Our evaluation shows that IDONTKNOWITSNAME...

In summary, we make the following contributions.

- We identify the inherent heterogeneity in programmatically generated prompts and its implications for serving.
- We design and build IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management....
- We conduct comprehensive evaluations of IDONTKNOWITSNAME against multiple established serving systems, demonstrating that IDONTKNOWITSNAME achieves up to...

2 Background

2.1 Serving systems for LLM agents

2.2 Agent frameworks with LLM calls as programming abstractions

3 Motivation

4 Design

In this section, we present the design of IDONTKNOWITSNAME, an efficient serving engine for program-integrated

LLM agents. IDONTKNOWITSNAME is a compiler-server co-design that exposes and exploits prompt heterogeneity for efficient KV-cache management.

On the compiler side, IDONTKNOWITSNAME performs static analysis over the agent program to expose the heterogeneity of prompts. The heterogeneity metadata is enveloped into a template descriptor. This descriptor includes agent application level structure and the heterogeneity within each LLM callsites. Server side then will consume this descriptor. First, IDONTKNOWITSNAME's combines runtime observations and the template descriptor to decide the optimal layout to expose longer prefix share to the underlying KV cache. Then, runtime logs are applied to predictively manage KV cache retention and eviction, Together with the metadata from the template descriptor.

4.1 Static Analysis for Template Derivation

The static pass of IDONTKNOWITSNAME's compiler analysis generates a set of per call-site template descriptors and a global program structure template. The per call-site template descriptor captures the heterogeneity of prompts at each LLM callsite, while the global program structure template captures the overall dataflow of the agent.

4.2 Generate call-site template

As we discussed in 3. Components within an programmatically generated prompt can be roughly divided into two categories: (i) **Invariant Specifications:** These are the components that is fixed once the program is compiled, such as system prompt, schema specifications, task descriptions. (ii) **Dynamic Bindings:** Because LLM calls are declared as functions, the parameter values are the actual user inputs. These runtime values are the dynamic bindings that are determined during runtime.

Therefore, we define our call-site template as:

$$\mathcal{T}_c = \langle \mathcal{I}_c, \mathcal{B}_c, \mathcal{A}_c \rangle.$$

Where $\mathcal{I}_c$

$$P_{c,t} = \text{Instantiate}(\mathcal{T}_c, vc, t; \pi_c),$$

5 Related Work

LLM programming abstractions. MTP and its ByLLM implementation derive LLM operations from typed declarations and semantic information [3]. DSPy separates declarative signatures and modules from prompt construction and optimization [9]. NVIDIA Object-Oriented Agents embeds LLM behavior in Python objects [5], while SIGIL compiles skill documents into typed harnesses with explicit control flow [4]. These systems make program structure available during application construction. IDONTKNOWITSNAME studies how the resulting structure can inform serving: named bindings expose value evolution and sharing that token-prefix lookup alone does not identify. It consumes supplied

templates or infers them from observed requests, and uses the recovered structure for cache management.

Memory-efficient inference and hierarchical caching. vLLM’s PagedAttention improves KV allocation and sharing through paged memory management [10]. SGLang combines a structured programming interface with RadixAttention and cache-aware scheduling [19]. Sarathi-Serve uses chunked prefill to improve the throughput–latency tradeoff [1]. CachedAttention retains conversation KV in a hierarchy and overlaps data movement with computation [6]; Mooncake builds a distributed cache and scheduling architecture around KV reuse [14]. These mechanisms provide allocation, lookup, transfer, and execution support. IDONTKNOWITSNAME builds on SGLang and adds decisions about the serialized prefix and its future demand, using binding lifetimes to determine what can be reused and prepared.

Modular and non-prefix attention reuse. Prompt Cache explicitly defines reusable prompt modules through schemas and manages their positions for attention-state reuse [7]. CacheBlend targets inputs assembled from multiple text chunks, reusing cached states beyond the common prefix and selectively recomputing tokens to restore cross-chunk interactions [15]. These works recognize that a prompt contains reusable components. IDONTKNOWITSNAME instead exposes common prefixes by reorganizing labelled bindings according to their observed evolution, then uses ordinary exact prefix caching on the resulting requests. It does not transplant KV tensors computed under a different preceding context. Exact cache reuse for a transformed request does not imply that the transformation preserves model outputs; the task-quality evaluation addresses that separate question.

Workflow-aware agent serving. KVFlow represents an agent schedule as an Agent Step Graph and uses distance to future execution for node-level eviction and overlapped prefetching [12]. PBKV predicts several future agent invocations in dynamic workflows using historical executions and current context, then conservatively guides eviction and prefetching [18]. CacheScout learns agent transitions online without a predefined workflow graph [17]. Continuum uses bounded retention through KV-cache time-to-live to preserve continuity across tool calls [11]. Pythia exposes workflow semantics through a serving interface to drive speculative cache management, lookahead scheduling, and autoscaling [16].

These systems establish the value of anticipating agent execution, including managing individual cache nodes and shared prefixes. IDONTKNOWITSNAME’s distinction is the coupling of that anticipation with the structure inside generated prompts. It learns which bindings are constant, copied, extended, or volatile; uses this information to change the reusable prefix; and reconstructs the known portion of predicted requests to guide preparation and retention. Thus,

workflow prediction supplies the likely destination and time of reuse, while binding analysis supplies the content and boundary of the reusable prefix.

6 Conclusion

The inherent structure and heterogeneity of programmatically generated prompts create opportunities for optimization. IDONTKNOWITSNAME identifies and exploits these opportunities rather than treating each prompt as a monolithic unit, thereby unlocking proactive optimizations that improve performance and efficiency. Our evaluation demonstrates that IDONTKNOWITSNAME’s methodology successfully creates a more efficient LLM serving system.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 117–134.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babushkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [3] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothynathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [4] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. arXiv:2607.27309 [cs.SE] <https://arxiv.org/abs/2607.27309>
- [5] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. arXiv:2607.20709 [cs.AI] <https://arxiv.org/abs/2607.20709>
- [6] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [7] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338.

- https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html
- [8] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [11] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230 [cs.OS]
- [12] Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400 [cs.DC] <https://arxiv.org/abs/2507.07400>
- [13] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 48371–48392.
- [14] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [15] Jiayi Yao, Hanchen Li, Yuhuan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 16 pages. doi:10.1145/3689031.3696098
- [16] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [17] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhuan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [18] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [19] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica,
- Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.